Doc. no.: P0317R1
Date: 2016-10-15
Reply to: Beman Dawes <bdawes at acm dot org>
Audience: Library

# Directory Entry Caching for Filesystem (R1)

**Fixing issue 2663, *Enable efficient retrieval of file size from directory_entry***
**and issue 2677, *directory_entry::status is not allowed to be cached ...***

This paper provides a solution to the problem of efficiently caching state information obtained during directory iteration.

The proposal has been implemented. Guidance is provided for users on how to use or not use cached information as desired, without exposing to the user whether information is actually cached. The proposal allows the user to write code that is fully portable between implementations that cache or do not cache.

## Background

Directory iteration in real-world operating systems always returns directory state information containing at least the file name. POSIX has an option to also return file status, but not all popular distributions implement this. Windows always returns file status, file size, and last modification date. Accessing this additional state from the directory entry is much more efficient than re-accessing the file system to obtain it. Users know this and expect the standard library filesystem to deliver the same efficiency.

The initial filesystem TS proposal and the Boost Filesystem implementation limited the additional state stored by class directory_entry to the regular status and symlink status, since these were the only additional elements common to several operating systems. The caching of this additional information was described using mutable exposition-only data members. The LWG removed the mutable members and associated caching wording because mutable members are problematic and race prone in multi-threaded environments. The original design also exposed too many implementation details and was not easily extendible to additional cache information such as file size and last write timestamp.

When directory_entry caching was removed from the TS, the LWG promised to revisit the issue when Filesystem was added to the standard. Two issues where subsequently filed that remind us of that promise.

> LWG issue 2663, *Enable efficient retrieval of file size from directory_entry*, requests that for Windows caching be extended to file size.
>
> LWG issue 2677, *directory_entry::status is not allowed to be cached as a quality-of-implementation issue*, requests the reinstatement of permission for implementations to cache directory entry information.

The Boost Filesystem library has implemented directory entry caching for many years. That implementation has been updated to conform to this proposal, and is passing its test suite. It will ship to users later this year.

## Design decisions

### Add cache refresh() functions

The original TS design conflated the observer functions that access cached state information with refreshing the cached state. Since refreshing the cached state is non-const, the cached member data had to be mutable and that was unacceptable. This proposal provides separate non-const refresh functions that are called by all other non-const functions that modify the stored path, ensuring cache integrity. The refresh functions can be called by users if desired to refresh stale cached data. With the separation of refresh and observer functionality, the observer functions become truly const.

### Provide observer functions for future needs

During LEWG discussions in Oulu, Geoffrey Romer suggested providing observer functions for attributes beyond those currently supported by real-world file systems. This has the effect of future-proofing directory_entry as file systems evolve. It also sparked the realization that the is_* family of filesystem query functions could be supported efficiently in a user-convenient way.

## Revision History

**R1 - pre-Issaquah:**

**Changes requested in Oulu**

- Select the first of the two approaches proposed in R0, per Oulu LEWG vote, enhanced with Geoffrey Romer's suggestion to provide observer functions for future needs, per Oulu LEWG vote.
- Change "for" to "such as" in [class.directory_entry], per Oulu LWG.
- Use endl rather than '\n' in example, per Oulu LWG.
- Clarify the relationship between directory iteration (which stores the cached values) and directory entries (which provide observers for the cached values and are where the cached values are stored). In reviewing the Oulu LWG discussion it became obvious that the split in caching responsibilities between the directory_entry and directory_iteration was not adequately described.
- Modify the [class.directory_entry] example to use explicit call to refresh().
- Reviewed all directory_entry member functions, per Oulu LWG, and add wording to *Returns* or other elements when needed to specify what happens when an error occurs. Note: If a *Returns* element is specified as simply calling a function whose returned value is well-specified as to the value returned on an error, there is no need to say anything further.
- Add an additional signature taking an error_code& argument to several functions, and updated specifications accordingly, per Oulu LWG.
- Add Acknowledgements.

**Changes requested in Chicago**

- Fix mispelling and spurious backslash per Walter Brown.
- After several flip-flops, Chicago LWG settles on "// exposition only" for "friend class directory_iterator;  // exposition only".
- Strike "() and change "the" to "any" to now read "any directory_entry refresh function" in the new paragraph being added to [class.directory_iterator], per Chicago LWG.
- Add [directory_entry.obs] introductory sentence per Chicago LWG.

**R0 - pre-Oulu:** Initial proposal providing two possible approaches to directory entry caching.


## Acknowledgements

Thanks to the LEWG and LWG for their many corrections and helpful suggestions. Thanks to Daniel Krügler for his pre-Chicago wording review. Special thanks to Geoffrey Romer for suggesting observer functions to meet future needs, resolving a major concern.


# Proposed wording

*Changes are relative to N4606.*

## 27.10.12 Class directory_entry [class.directory_entry]

```
namespace std::filesystem {
 class directory_entry {
 public:
  // constructors and destructor
  directory_entry() noexcept = default;
  directory_entry(const directory_entry&) = default;
  directory_entry(directory_entry&&) noexcept = default;
  explicit directory_entry(const path& p);
  directory_entry(const path& p, error_code& ec);
 ~directory_entry();

  // assignments
  directory_entry& operator=(const directory_entry&) = default;
  directory_entry& operator=(directory_entry&&) noexcept = default;

  // modifiers
  void assign(const path& p);
  void assign(const path& p, error_code& ec);
  void replace_filename(const path& p);
  void replace_filename(const path& p, error_code& ec);
  void refresh();
  void refresh(error_code& ec) noexcept;

  // observers
  const path&  path() const noexcept;
  operator const path&() const noexcept;
```

```
  bool exists() const;
  bool exists(error_code& ec) const noexcept;
  bool is_block_file() const;
  bool is_block_file(error_code& ec) const noexcept;
  bool is_character_file() const;
  bool is_character_file(error_code& ec) const noexcept;
  bool is_directory() const;
  bool is_directory(error_code& ec) const noexcept;
  bool is_fifo() const;
  bool is_fifo(error_code& ec) const noexcept;
  bool is_other() const;
  bool is_other(error_code& ec) const noexcept;
  bool is_regular_file() const;
  bool is_regular_file(error_code& ec) const noexcept;
  bool is_socket() const;
  bool is_socket(error_code& ec) const noexcept;
  bool is_symlink() const;
  bool is_symlink(error_code& ec) const noexcept;
  uintmax_t file_size() const;
  uintmax_t file_size(error_code& ec) const noexcept;
  uintmax_t hard_link_count() const;
  uintmax_t hard_link_count(error_code& ec) const noexcept;
  file_time_type last_write_time() const;
  file_time_type last_write_time(error_code& ec) const noexcept;
  file_status status() const;
  file_status status(error_code& ec) const noexcept;
  file_status symlink_status() const;
  file_status symlink_status(error_code& ec) const noexcept;

  bool operator< (const directory_entry& rhs) const noexcept;
  bool operator==(const directory_entry& rhs) const noexcept;
  bool operator!=(const directory_entry& rhs) const noexcept;
  bool operator<=(const directory_entry& rhs) const noexcept;
  bool operator> (const directory_entry& rhs) const noexcept;
  bool operator>=(const directory_entry& rhs) const noexcept;
 private:
  path   pathobject; // exposition only
  friend class directory_iterator;  // exposition only
 };

}
```

A directory_entry object stores a path object and may store additional objects for file attributes such as hard link count, status, symlink status, file size, and last write time.

Implementations are encouraged to store such additional file attributes during directory iteration if their values are available and storing the values would allow the implementation to eliminate file system accesses by directory_entry observer functions ([fs.op.funcs]). Such stored file attribute values are said to be *cached*.

[*Note:* For purposes of exposition, class directory_iterator ([class.directory_iterator]) is shown above as a friend of class directory_entry. Friendship allows the directory_iterator implementation to cache already available attribute values directly into a directory_entry object without the cost of an unneeded call to refresh(). —*end note*]

[*Example:*

```
using namespace std::filesystem;

// use possibly cached last write time to minimize disk accesses
for (auto&& x : directory_iterator("."))
{
  std::cout << x.path() << " " << x.last_write_time() << std::endl;
}

// call refresh() to refresh a stale cache
for (auto&& x : directory_iterator("."))
{
  lengthy_function(x.path());  // cache becomes stale
  x.refresh();
  std::cout << x.path() << " " << x.last_write_time() << std::endl;
}
```

On implementations that do not cache the last write time, both loops will result in a potentially expensive call to the std::filesystem::last_write_time function.

On implementations that do cache the last write time, the first loop will use the cached value and so will not result in a potentially expensive call to the std::filesystem::last_write_time function.

The code is portable to any implementation, regardless of whether or not it employs caching.

*—end example*]

### 27.10.12.1 directory_entry constructors [directory_entry.cons]

```
explicit directory_entry(const path& p);
directory_entry(const path& p, error_code& ec);
```

*Effects:* Constructs an object of type directory_entry, then refresh()or refresh(ec), respectively.

*Postcondition:* path() == p if no error occurs, otherwise path() == std::filesystem::path().

*Throws:* As specified in Error reporting ([fs.err.report]).

### 27.10.12.2 directory_entry modifiers [directory_entry.mods]

```
void assign(const path& p);
void assign(const path& p, error_code& ec);
```

*Effects:* Equivalent to pathobject = p, then refresh()or refresh(ec), respectively. If an error occurs the value of any cached attributes is unspecified.

*Postcondition:* path() == p.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
void replace_filename(const path& p);
void replace_filename(const path& p, error_code& ec);
```

*Effects:* Equivalent to pathobject = pathobject.parent_path()/ p, then refresh()or refresh(ec), respectively. If an error occurs the value of any cached attributes is unspecified.

*Postcondition:* path() == x.parent_path() / p where x is the value of path() before the function is called.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
void refresh();
void refresh(error_code& ec) noexcept;
```

*Effects:* Stores the current value for any cached attribute values of the file p resolves to. If an error occurs, an error is reported (27.10.7 [fs.err.report]) and the value of any cached attributes is unspecified.

*Throws:* As specified in Error reporting ([fs.err.report]).

[*Note*: Implementations of directory_iterator [class.directory_iterator] are prohibited from directly or indirectly calling the refresh function since it must access the external file system, and the objective of caching is to avoid unnecessary file system accesses. *—end note*]

### 27.10.12.3 directory_entry observers [directory_entry.obs]

directory_entry observers described below as returning a function with an unqualified name shall behave as if calling the namespace std::filesystem non-member function of that name.

```
const path& path() const noexcept;
operator const path&() const noexcept;
```

*Returns:* pathobject

```
bool exists() const;
bool exists(error_code& ec) const noexcept;
```

*Returns:* exists(this->status()) or exists(this->status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool is_block_file() const;
bool is_block_file(error_code& ec) const noexcept;
```

*Returns:* is_block_file(this->status()) or is_block_file(this->status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool is_character_file() const;
bool is_character_file(error_code& ec) const noexcept;
```

*Returns:* is_character_file(this->status()) or is_character_file(this->status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool is_directory() const;
bool is_directory(error_code& ec) const noexcept;
```

*Returns:* is_directory(this->file_status()) or is_directory(this->file_status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool is_fifo() const;
bool is_fifo(error_code& ec) const noexcept;
```

*Returns:* is_fifo(this->status()) or is_fifo(this->status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool is_other() const;
bool is_other(error_code& ec) const noexcept;
```

*Returns:* is_other(this->status()) or is_other(this->status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool is_regular_file() const;
bool is_regular_file(error_code& ec) const noexcept;
```

*Returns:* is_regular_file(this->status()) or is_regular_file(this->status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool is_socket() const;
bool is_socket(error_code& ec) const noexcept;
```

*Returns:* is_socket(this->status()) or is_socket(this->status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool is_symlink() const;
bool is_symlink(error_code& ec) const noexcept;
```

*Returns:* is_symlink(this->symlink_status()) or is_symlink(this->symlink_status(ec)), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
uintmax_t file_size() const;
uintmax_t file_size(error_code& ec) const noexcept;
```

*Returns:* If cached, the file size attribute value. Otherwise, file_size(path()) or file_size(path(), ec), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
uintmax_t hard_link_count() const;
uintmax_t hard_link_count(error_code& ec) const noexcept;
```

*Returns:* If cached, the hard link count attribute value. Otherwise, hard_link_count(path()) or hard_link_count(path(), ec), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
file_time_type last_write_time() const;
file_time_type last_write_time(error_code& ec) const noexcept;
```

*Returns:* If cached, the last write time attribute value. Otherwise, last_write_time(path()) or last_write_time(path(), ec), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
file_status status() const;
file_status status(error_code& ec) const noexcept;
```

*Returns:* If cached, the status attribute value. Otherwise, status(path()) or status(path(), ec), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
file_status symlink_status() const;
file_status symlink_status(error_code& ec) const noexcept;
```

*Returns:* If cached, the symlink status attribute value. Otherwise, symlink_status(path()) or symlink_status(path(), ec), respectively.

*Throws:* As specified in Error reporting ([fs.err.report]).

```
bool operator==(const directory_entry& rhs) const noexcept;
```

*Returns:* pathobject == rhs.pathobject.

```
bool operator!=(const directory_entry& rhs) const noexcept;
```

*Returns:* pathobject != rhs.pathobject.

```
bool operator< (const directory_entry& rhs) const noexcept;
```

*Returns:* pathobject < rhs.pathobject.

```
bool operator<=(const directory_entry& rhs) const noexcept;
```

*Returns:* pathobject <= rhs.pathobject.

```
bool operator> (const directory_entry& rhs) const noexcept;
```

*Returns:* pathobject > rhs.pathobject.

```
bool operator>=(const directory_entry& rhs) const noexcept;
```

*Returns:* pathobject >= rhs.pathobject.

### 7.10.13 Class directory_iterator [class.directory_iterator]

An object of type directory_iterator provides an iterator for a sequence of directory_entry elements representing the files path and any cached attribute values ([class.directory_entry]) for each file in a directory. [ *Note:* For iteration into sub-directories, see class recursive_directory_iterator (27.10.14). —*end note* ]

*Insert a new paragraph before the current note that begins at paragraph 9:*

Constructors and non-const directory_iterator member functions shall store the cached attribute values, if any, described in ([class.directory_entry]) in the directory_entry element returned by operator*(). directory_iterator member functions shall not directly or indirectly call any directory_entry refresh function.[*Note:* The exact mechanism for storing cached attribute values is not exposed to users. For exposition class directory_iterator is shown in [class.directory_entry] as a friend of class directory_entry. —*end note*]

---

## Note for implementors

For Windows, directory_entry information is obtained during directory iteration by calling function FindFirstFile, FindFirstFileEx, or FindNextFile, and can be refreshed by calling function GetFileInformationByHandle or GetFileAttributesEx.

For some POSIX-like systems, such as Linux and some BSD-based distributions, glibc versions since 2.19 may support an additional struct dirent field named d_type "making it possible to avoid the expense of calling lstat". POSIX specifies that the macro _DIRENT_HAVE_D_TYPE is defined if d_type is present.

## References


Issue 2663, *Enable efficient retrieval of file size from directory_entry*, cplusplus.github.io/LWG/lwg-active.html#2663

Issue 2677, *directory_entry::status is not allowed to be cached as a quality-of-implementation issue*, cplusplus.github.io/LWG/lwg-active.html#2677

N4582, *Working Draft, Standard for Programming Language C++, 2016*, www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4582.pdf

N4100, *Programming Languages — C++ — File System Technical Specification, 2014*, www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4100.pdf

*Boost Filesystem Library, V3*, 2015, www.boost.org/doc/libs/1_60_0/libs/filesystem/doc/index.htm


---